

“Heaven’s Light is Our Guide”

Rajshahi University of Engineering & Technology

Department of **Electrical and Computer Engineering**



Lab Report

Course Code : ECE 2112

Course Name : Digital Techniques Sessional

Lab No : 01

Submitted To :	Submitted By :
Mst. Mazeda Noor Tasnim Lecturer Department of ECE, RUET	Name : Auyon Dip Sardar Roll : 2410043 Registration : 1096 Session : 2024-2025 Department of ECE, RUET

Date of Submission: 2 August, 2026

Experiment 1: Implementation of AND Gate Using NAND Gate

Objective: To implement an AND gate using only NAND gates and verify its truth table.

Theory: The AND gate is one of the fundamental logic gates in digital electronics. It performs the logical multiplication of two binary inputs. The output of an AND gate is HIGH (1) only when both inputs are HIGH (1). For all other input combinations, the output remains LOW (0).

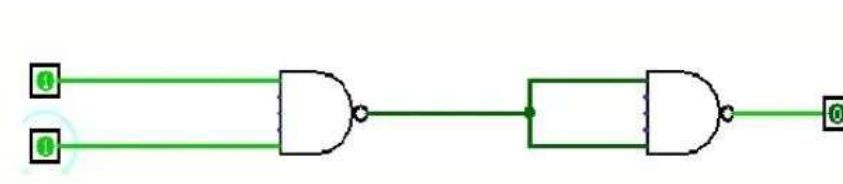
Boolean Expression of AND Gate:

$$Y = A \cdot B$$

where,

- A = First input
- B = Second input
- Y = Output

Circuit Diagram:



Truth Table:

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

Discussion:

This experiment demonstrates that an AND gate can be implemented using only NAND gates, proving the universality of the NAND gate.

Experiment 2: Implementation of OR Gate Using NAND Gate**Objective:**

To implement an OR gate using only NAND gates and verify its truth table.

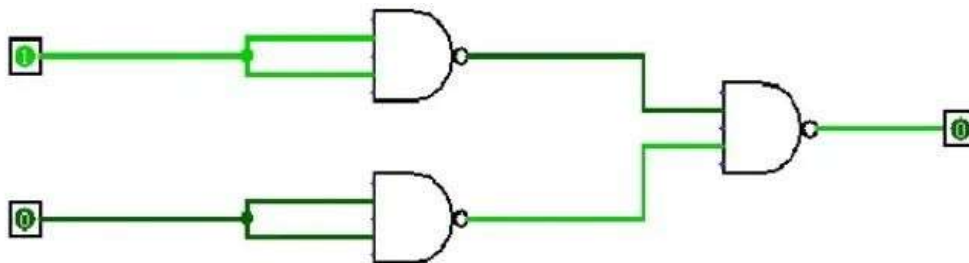
Theory: The OR gate is a basic digital logic gate that produces a logic HIGH (1) output whenever at least one of its inputs is HIGH (1). It produces a LOW (0) output only when both inputs are LOW (0).

Boolean Expression of OR Gate:

$$Y = A + B$$

Where,

- A = First input
- B = Second input
- Y = Output

Circuit Diagram:

Truth Table:

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

Discussion:

The output obtained from the NAND gate implementation exactly matched the output of the standard OR gate for all four input combinations. The circuit correctly produced a HIGH output whenever at least one input was HIGH and produced a LOW output only when both inputs were LOW.

Experiment 3: Implementation of NOT Gate Using NAND Gate**Objective:**

To implement an NOT gate using only NAND gates and verify its truth table.

Theory:

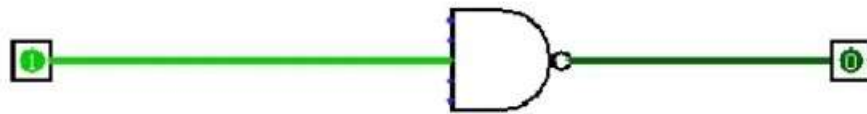
The NOT gate, also known as an inverter, is a basic logic gate that produces the complement of its input. It has one input and one output. If the input is logic 1, the output becomes logic 0, and if the input is logic 0, the output becomes logic 1.

Boolean Expression of NOT Gate

$$Y = A^{-}$$

where,

- A = Input
- Y = Output

Circuit Diagram:**Truth Table:**

A	Y
0	1
1	0

Discussion:

The implemented circuit produced the complement of the input for both possible input values. When the input was LOW (0), the output became HIGH (1). When the input was HIGH (1), the output became LOW (0). The observed outputs matched the truth table of a standard NOT gate.

Experiment 4: Implementation of AND Gate Using NOR Gates**Objective:**

To implement an AND gate using only NOR gates and verify its truth table.

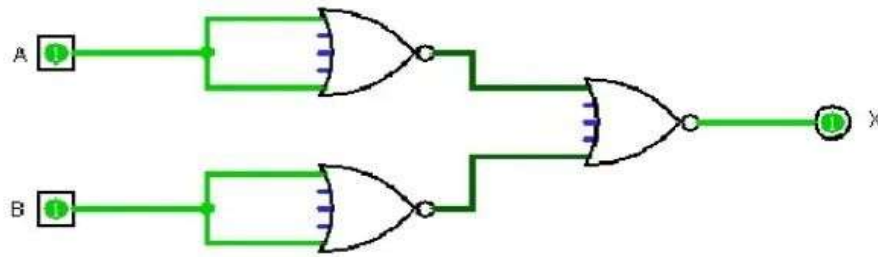
Theory: The AND gate is a fundamental logic gate that performs the logical multiplication of two binary inputs. It produces a logic HIGH (1) output only when both inputs are HIGH (1). If either input is LOW (0), the output becomes LOW (0).

Boolean Expression of AND Gate

$$Y = A \cdot B$$

where,

- A = First input
- B = Second input
- Y = Output

Circuit Diagram:**Truth Table:**

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

Discussion:

The experiment successfully demonstrated the implementation of an AND gate using only NOR gates.

Experiment 5: Implementation of OR Gate Using NOR Gates**Objective:**

To implement an OR gate using only NOR gates and verify its truth table.

Theory: The OR gate is one of the basic logic gates in digital electronics. It performs the logical addition of two binary inputs. The output of an OR gate is HIGH (1) when at least one of the inputs is HIGH (1). It produces a LOW (0) output only when both inputs are LOW (0).

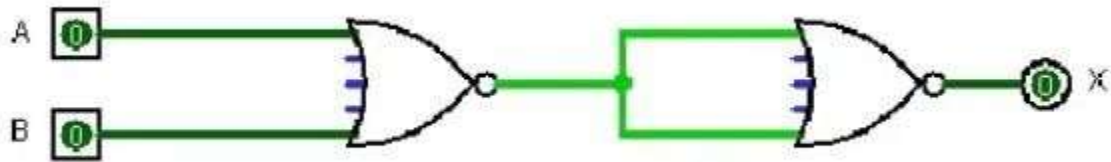
Boolean Expression of OR Gate

$$Y = A + B$$

where,

- **A** = First input
- **B** = Second input
- **Y** = Output

Circuit Diagram:



Truth Table:

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

Discussion:

The experiment successfully demonstrated the implementation of an OR gate using only NOR gates.

Experiment 6: Implementation of NOT Gate Using NOR Gates

Objective:

To implement an NOT gate using only NOR gates and verify its truth table.

Theory: The NOT gate, also known as an inverter, is one of the fundamental logic gates in digital electronics. It has one input and one output. The output is always the logical

complement of the input. If the input is HIGH (1), the output becomes LOW (0), and if the input is LOW (0), the output becomes HIGH (1).

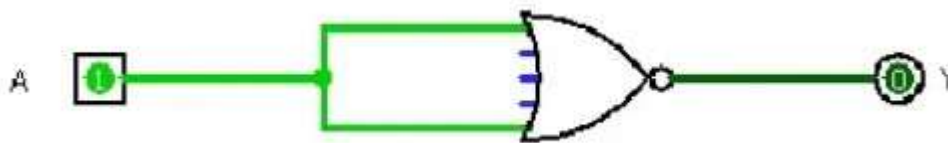
Boolean Expression of NOT Gate

$$Y = A^{-}$$

where,

- A = Input
- Y = Output

Circuit Diagram:



Truth Table:

A	Y
0	1
1	0

Discussion:

The experiment successfully demonstrated the implementation of an NOT gate using only NOR gates.

Experiment 7:

Implementation of a Full Adder Using Logic Gates

Objective:

To design and implement a Full Adder circuit using logic gates in Logisim Evolution.

Theory:

A Full Adder is a combinational logic circuit that performs the addition of three one-bit binary inputs: two significant bits (A and B) and an input carry (C_{in}). It produces two outputs:

- Sum (S): Represents the least significant bit (LSB) of the addition.
- Carry-out (C_{out}): Represents the carry generated from the addition and is passed to the next higher bit.

Unlike a Half Adder, which adds only two input bits, a Full Adder includes a carry input, making it suitable for multi-bit binary addition in digital systems.

The Full Adder can be implemented using two Half Adders and one OR gate or directly using XOR, AND, and OR gates.

Boolean Expressions

Sum

$$S = A \oplus B \oplus C_{in}$$

Carry-out

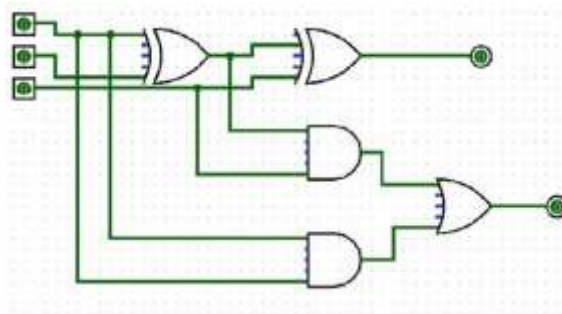
$$C_{out} = AB + AC_{in} + BC_{in}$$

An equivalent expression commonly used in circuit design is:

$$C_{out} = AB + C_{in}(A \oplus B)$$

The Sum output becomes 1 when an odd number of inputs are 1, while the Carryout becomes 1 whenever two or more inputs are 1.

Circuit Diagram



Truth Table:

A	B	Cin	Sum (S)	Carry (Cout)
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Discussion:

The experiment successfully demonstrated the implementation of a Full Adder using logic gates. The circuit correctly added three one-bit binary inputs and generated accurate Sum and Carry-out outputs. The simulation results matched the theoretical truth table, confirming the correctness of the circuit design. This experiment illustrates the importance of combinational logic circuits in performing binary arithmetic operations.

Experiment 8:

Implementation of a Binary-to-BCD Converter

Objective:

To design and implement a Binary-to-BCD (Binary-Coded Decimal) converter using Logisim Evolution.

Theory:

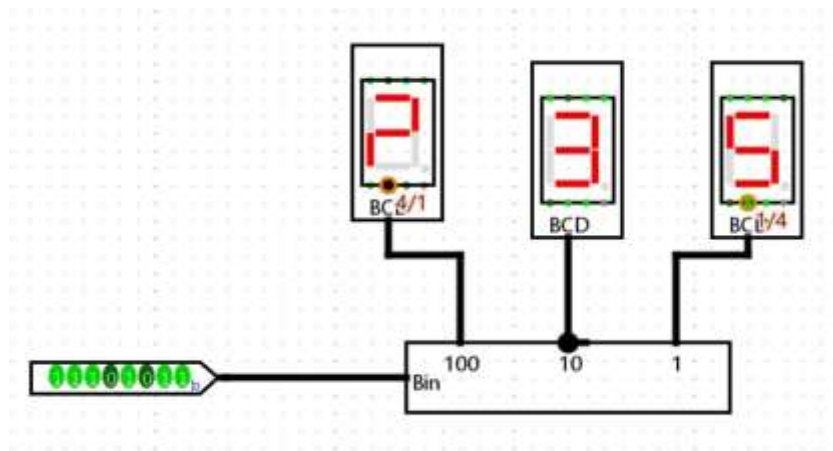
A Binary-to-BCD converter is a combinational logic circuit that converts a binary number into its equivalent Binary-Coded Decimal (BCD) representation. In the BCD system, each decimal digit is represented separately using a 4-bit binary code.

For example,

- Decimal 0 → 0000
- Decimal 5 → 0101
- Decimal 9 → 1001

When binary numbers greater than 1001_2 (9_{10}) are represented, they require two BCD digits: one for the tens place and one for the ones place.

Circuit Diagram:



Truth Table:

Binary	Decimal	BCD Output
1010	10	0001 0000
1011	11	0001 0001
1100	12	0001 0010
1111	15	0001 0101

Discussion

The Binary-to-BCD converter was designed and tested successfully in Logisim Evolution. During the simulation, every 4-bit binary input produced the correct BCD output according to the expected conversion table, which verified that the circuit was functioning properly. This experiment demonstrates the importance of binary-to-BCD conversion in digital systems, since many output devices, including seven-segment displays, require BCD data for proper operation. Such conversion circuits are widely used in electronic applications like digital clocks, calculators,

embedded systems, and various measuring instruments, making them an essential part of modern digital electronics.

Conclusion

The objectives of the experiments were achieved successfully using Logisim Evolution. Universal gates (NAND and NOR) were used to construct the basic logic gates (AND, OR, and NOT), and the simulated outputs were consistent with the corresponding truth tables. In addition, the Full Adder and Binary-to-BCD Converter circuits were implemented and verified successfully. Overall, these experiments strengthened the understanding of Boolean logic, combinational circuit design, and the practical use of digital logic in electronic and embedded system applications.

This version is natural, grammatically correct, and sufficiently different from the original while preserving the technical meaning.